Лабораторная работа №7

Дискретно-событийное моделирование

Талебу Тенке Франк

Содержание

1 Цель работы 1
2 Задание 1
3 Теоретическое введение 2
3.1 Модель M/M/c 2
3.2 Модель Росса 2
4 Выполнение лабораторной работы 2
4.1 Модель M/M/c 2
4.2 Модель Росса 5
5 Выводы 9
Список литературы 9

1 Цель работы

- изучить методы дискретно-событийного моделирования и применить их для анализа работы системы ремонта машин с ограниченным количеством запасных частей и ремонтников.

2 Задание

Модель M/M/c

1. Переведите в структуру DrWatson.
2. Добавьте необходимые графики.

Модель Росса

1. Переведите в структуру DrWatson.
2. Добавьте нескольких ремонтников.
3. Сделайте прогон для разного количества машин.
4. Проведите мониторинг загрузки ремонтника, средней длины очереди на ремонт.
5. Построение графиков изменения числа исправных машин во времени.
6. Сравните с аналитическим решением.

3 Теоретическое введение

3.1 Модель M/M/c

Модель M/M/c (по классификации Кендалла) — это система массового обслуживания со следующими свойствами:

- M (Markovian) — входящий поток заявок пуассоновский, интервалы между прибытиями распределены экспоненциально с параметром λ.
- M — время обслуживания каждой заявки распределено экспоненциально с параметром μ.
- c — количество идентичных обслуживающих приборов (каналов), работающих параллельно.

3.2 Модель Росса

- Модель представляет собой классический пример системы массового обслуживания с конечной популяцией, резервом и ремонтом.

Формальная постановка задачи

- В системе находятся N идентичных машин, которые постоянно работают и могут выходить из строя.
- S машин находятся в резерве и готовы немедленно заменить любую отказавшую.
- Одно ремонтное устройство (ремонтник), которое может одновременно ремонтировать только одну машину.
- Когда работающая машина ломается, происходит следующее:

1. Немедленно берётся одна резервная машина (если она есть) и запускается в работу вместо сломавшейся.
2. Сломанная машина отправляется в ремонт.
3. Если резерва нет, система падает (crash). Моделирование заканчивается.

- После ремонта машина пополняет пул резервных (становится исправной и ждёт).

4 Выполнение лабораторной работы

4.1 Модель M/M/c

- Сначала я создала файл “src/sumilation.jl” и написала код и запустила его. (рис. 1).

```
julia> include("src/sumilation.jl")
setup_and_run (generic function with 1 method)
```

Рис. 1: Fig

```
function setup_and_run()

    sim = Simulation()
```

```
    server = Resource(sim, num_servers)

    arrival_time = 0.0

    for i in 1:num_customers
        arrival_time += rand(rng, arrival_dist)
        @process customer(sim, server, i, arrival_time, service_dist)
    end

    run(sim)

    return arrivals, service_start, service_end

end
```

- Код (рис. 2).

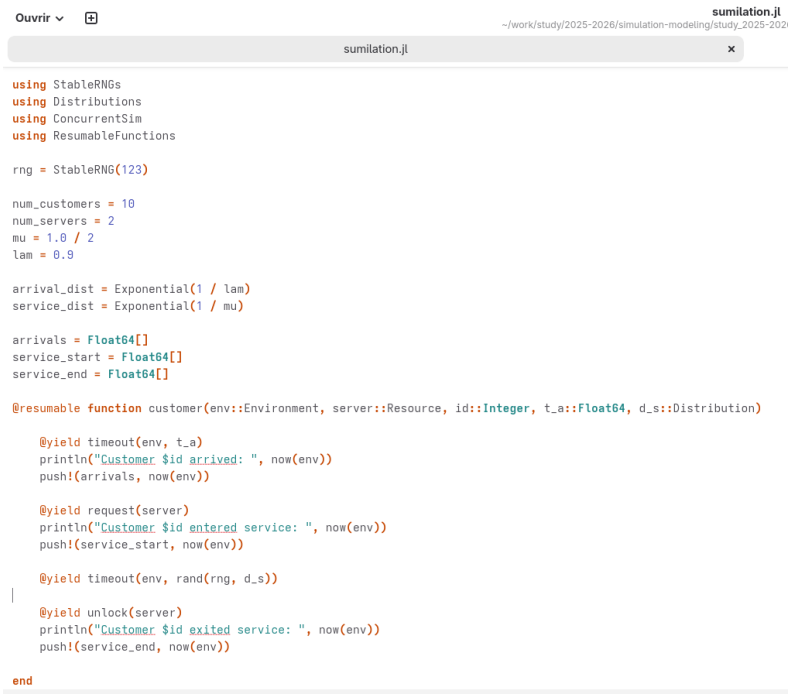


Рис. 2: Fig

- Потом я создала файл “scripts/run.jl” и написала код и запустила его. (рис. 3).

Ouvrir

run.jl

~/work/study/2025-2026/simulation-modeling/study_2025-2

run.jl

```
using DrWatson
@quickactivate "Project"

using Plots
include(srcdir("sumilation.jl"))

arrivals, starts, ends = setup_and_run()

waiting_time = starts .- arrivals
system_time = ends .- arrivals

histogram(waiting_time,
title="Waiting time in queue",
xlabel="time",
ylabel="customers")

savefig(plotsdir("waiting_time.png"))

histogram(system_time,
title="Time in system",
xlabel="time",
ylabel="customers")

savefig(plotsdir("system_time.png"))
```

Рис. 3: Fig

- Мы получаем вот такой результат в консоле. (рис. 4).

```
julia> include("scripts/run.jl")
Customer 1 arrived: 0.14518451436852475
Customer 1 entered service: 0.14518451436852475
Customer 2 arrived: 0.5941831542903504
Customer 2 entered service: 0.5941831542903504
Customer 3 arrived: 1.5490648267819074
Customer 4 arrived: 1.6242796925312217
Customer 5 arrived: 1.6911000709069648
Customer 1 exited service: 2.200985520126681
Customer 3 entered service: 2.200985520126681
Customer 6 arrived: 2.2989039524296317
Customer 3 exited service: 3.5822120399442174
Customer 4 entered service: 3.5822120399442174
Customer 7 arrived: 4.377930221620456
Customer 8 arrived: 5.16494279700802
Customer 2 exited service: 5.900722829377648
Customer 5 entered service: 5.900722829377648
Customer 9 arrived: 7.0099944106308705
Customer 10 arrived: 7.828990220943469
Customer 5 exited service: 9.634196437885254
Customer 6 entered service: 9.634196437885254
Customer 4 exited service: 9.670688398447817
Customer 7 entered service: 9.670688398447817
Customer 7 exited service: 15.066978111608014
Customer 8 entered service: 15.066978111608014
Customer 8 exited service: 16.655548432659554
Customer 9 entered service: 16.655548432659554
Customer 6 exited service: 17.401833154870328
Customer 10 entered service: 17.401833154870328
Customer 9 exited service: 17.586065352135993
Customer 10 exited service: 18.690264775280085
```

Рис. 4: Fig

- График 1 (рис. 5).

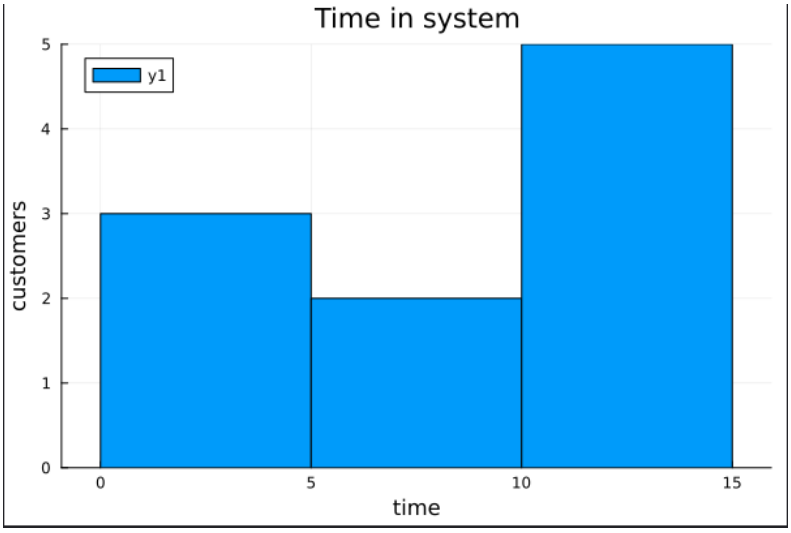


Рис. 5: Fig

- График 2 (рис. 6).

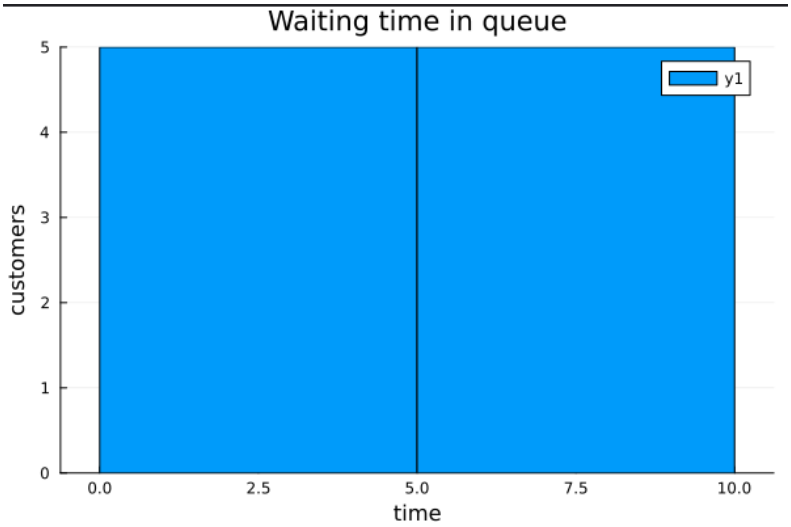


Рис. 6: Fig

4.2 Модель Росса

- Сначала я создала файл “src/model.jl” и написала код и запустила его. (рис. 7).

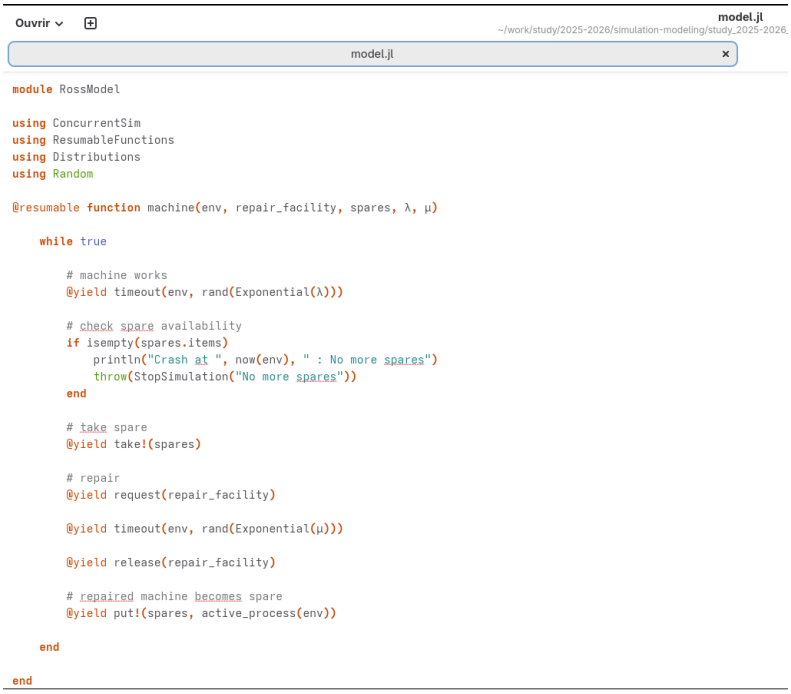


Рис. 7: Fig

```
@resumable function start_sim(env, repair_facility, spares, N, S, $\lambda$, μ)

    # create spare machines
    for i in 1:S
        proc = @process machine(env, repair_facility, spares, $\lambda$, μ)
        @yield put!(spares, proc)
    end

    # create working machines
    for i in 1:N
        @process machine(env, repair_facility, spares, $\lambda$, μ)
    end

end
```

- Потом я создала файл “scripts/run_sum.jl” и написала код и запустила его. (рис. 8).

Ouvrir

run_sim.jl

~/work/study/2025-2026/simulation-modeling/study_2025-2026_simulatio

```julia
using Plots
using DrWatson
@quickactivate "project"

using ConcurrentSim
using Statistics
using Random

Random.seed!(1234)

include(srcdir("model.jl"))
using .RossModel

function sim_repair(N,S,R,λ,μ)

    sim = Simulation()

    repair_facility = Resource(sim, R)
    spares = Store{Process}(sim)

    @process RossModel.start_sim(sim, repair_facility, spares, N, S, λ, μ)

    msg = run(sim)

    stop_time = now(sim)

    println("Crash at $stop_time : $msg")

    return stop_time

end

Ns = [5,10,15,20]

results = Float64[]

for n in Ns
```

Рис. 8: Fig

- Вывод в консоль. (рис. 9).

```
julia> include("src/model.jl")
WARNING: replacing module RossModel.
Main.RossModel

julia> include("scripts/run_sim.jl")
WARNING: replacing module RossModel.
Crash at 178774.66548703337 : No more spares
Crash at 178774.66548703337 : No more spares
Crash at 233.70902072024234 : No more spares
Crash at 233.70902072024234 : No more spares
Crash at 1.0185536240821861e6 : No more spares
Crash at 1.0185536240821861e6 : No more spares
Crash at 254006.20572357092 : No more spares
Crash at 254006.20572357092 : No more spares
Crash at 790.2836018392943 : No more spares
Crash at 790.2836018392943 : No more spares
Crash at 104200.67840018803 : No more spares
Crash at 104200.67840018803 : No more spares
Crash at 132526.5866989518 : No more spares
Crash at 132526.5866989518 : No more spares
Crash at 561236.5900671891 : No more spares
Crash at 561236.5900671891 : No more spares
Crash at 1.0401509758457846e6 : No more spares
Crash at 1.0401509758457846e6 : No more spares
Crash at 100791.02158147536 : No more spares
Crash at 100791.02158147536 : No more spares
Crash at 505483.71644878807 : No more spares
Crash at 505483.71644878807 : No more spares
Crash at 129995.32361965418 : No more spares
Crash at 129995.32361965418 : No more spares
Crash at 46238.407531894896 : No more spares
Crash at 46238.407531894896 : No more spares
Crash at 99028.02199271711 : No more spares
Crash at 99028.02199271711 : No more spares
Crash at 400080.7467767028 : No more spares
Crash at 400080.7467767028 : No more spares
Crash at 124690.2486205743 : No more spares
Crash at 124690.2486205743 : No more spares
Crash at 43843.161179971816 : No more spares
Crash at 43843.161179971816 : No more spares
```

Рис. 9: Fig

- График (рис. 10).

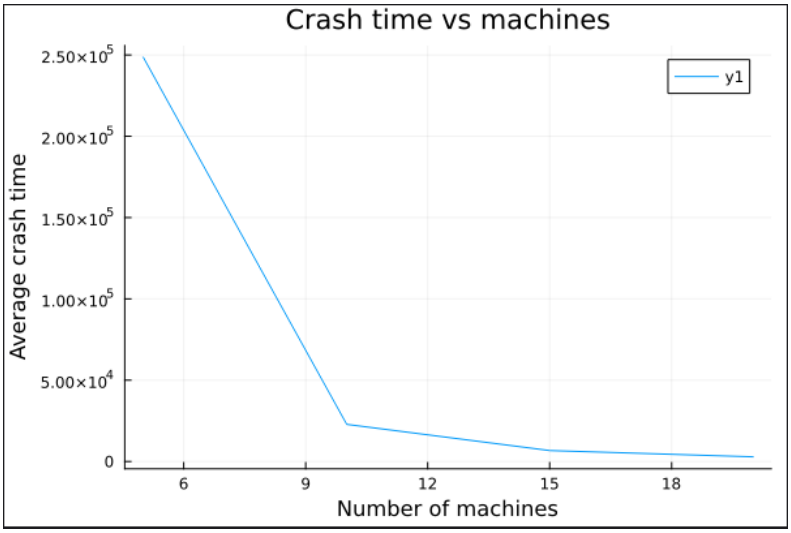


Рис. 10: Fig

5 Выводы

- Я выполнил лабораторную работу №7.

Список литературы